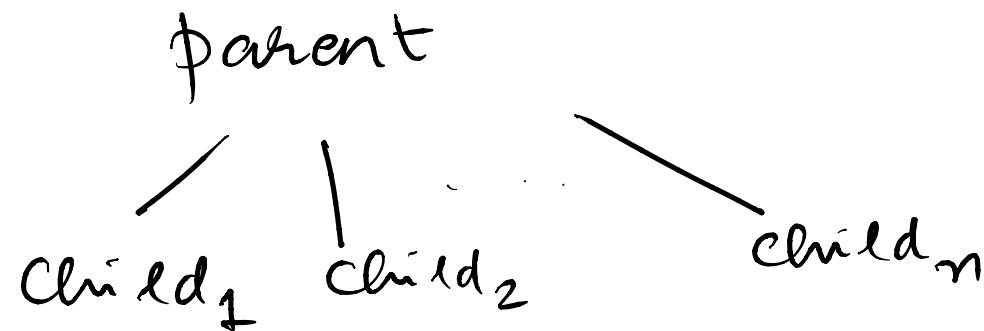


Trees

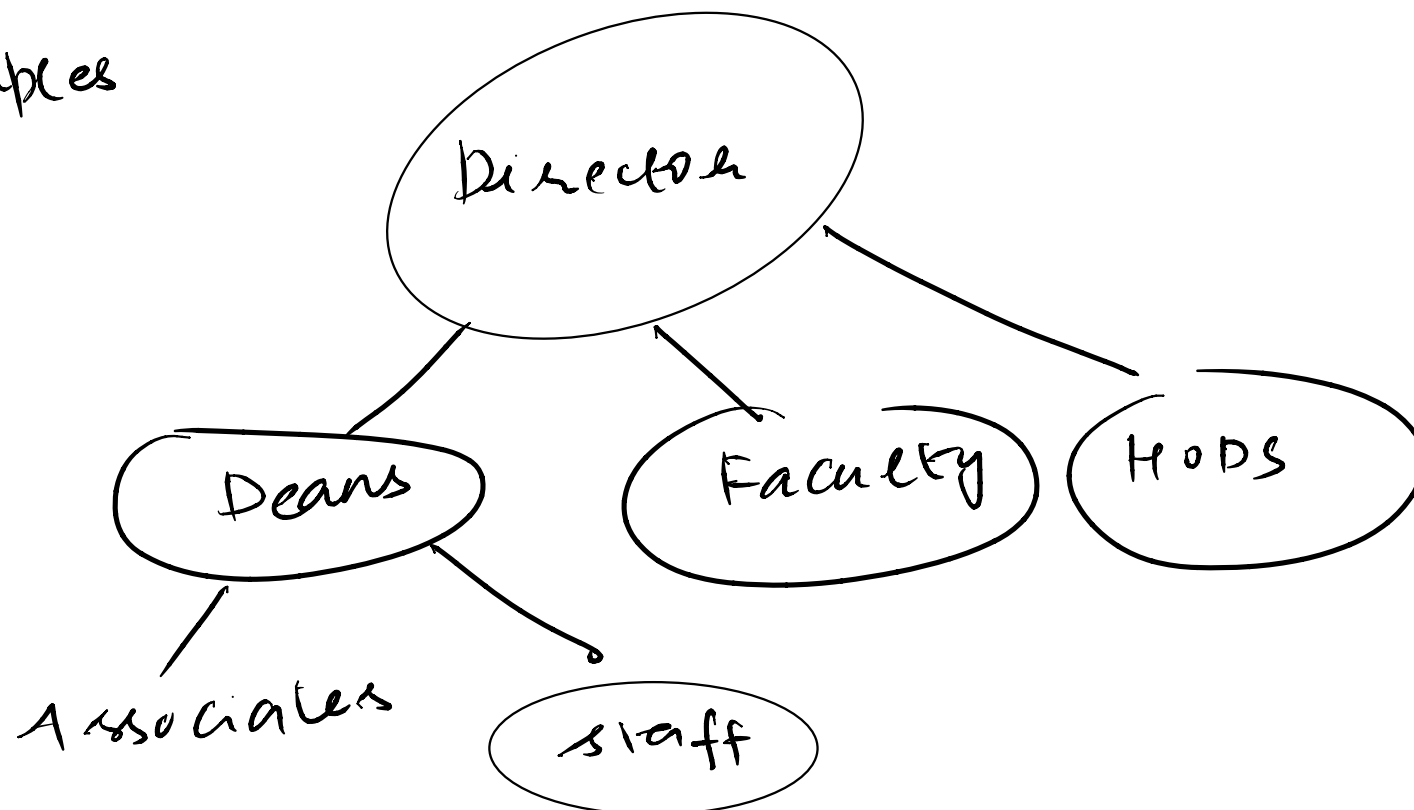
Hierarchical Structures

Number of
children
vary across nodes

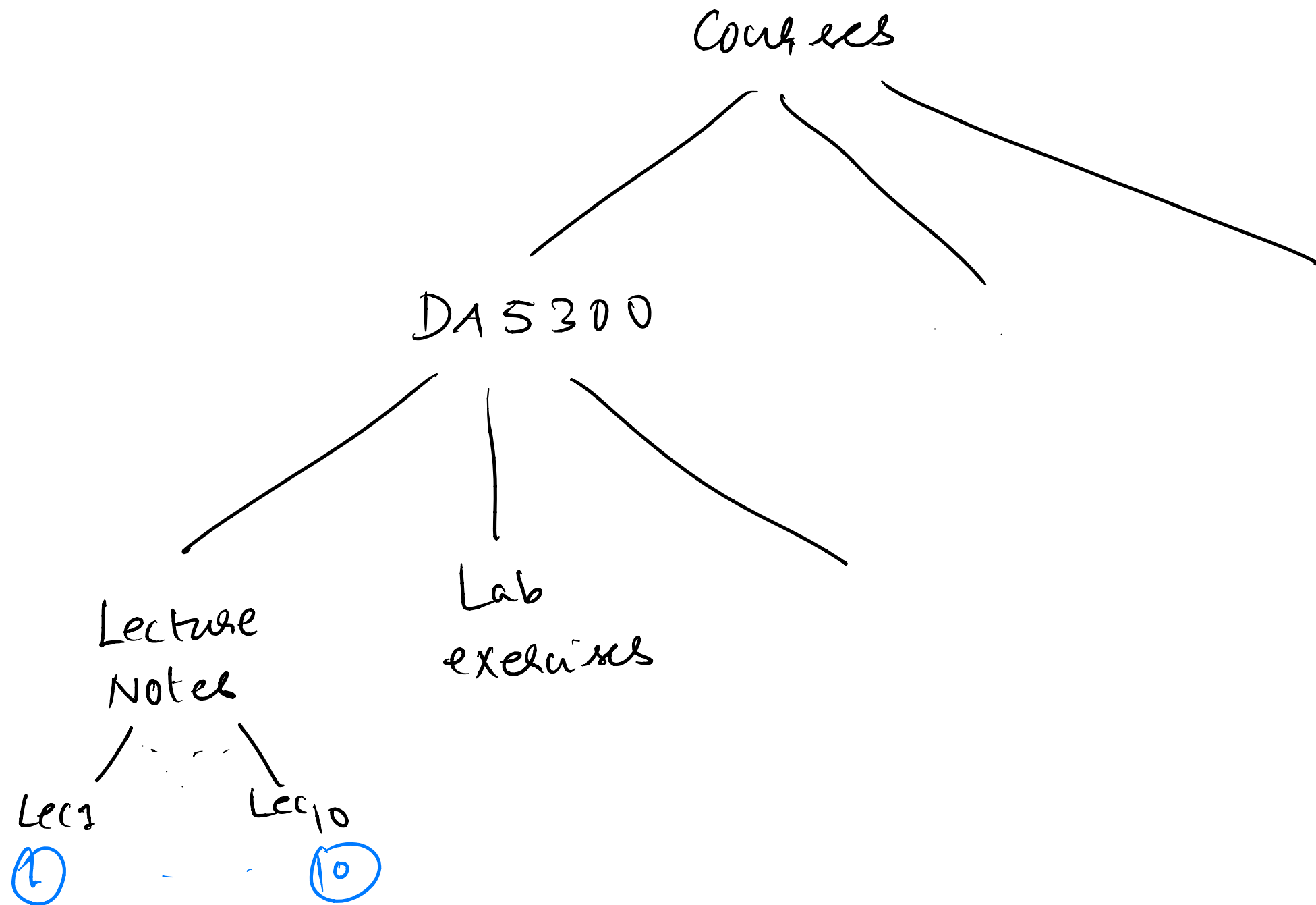


} this can be recursive in that children can have their own children etc

Practical Examples

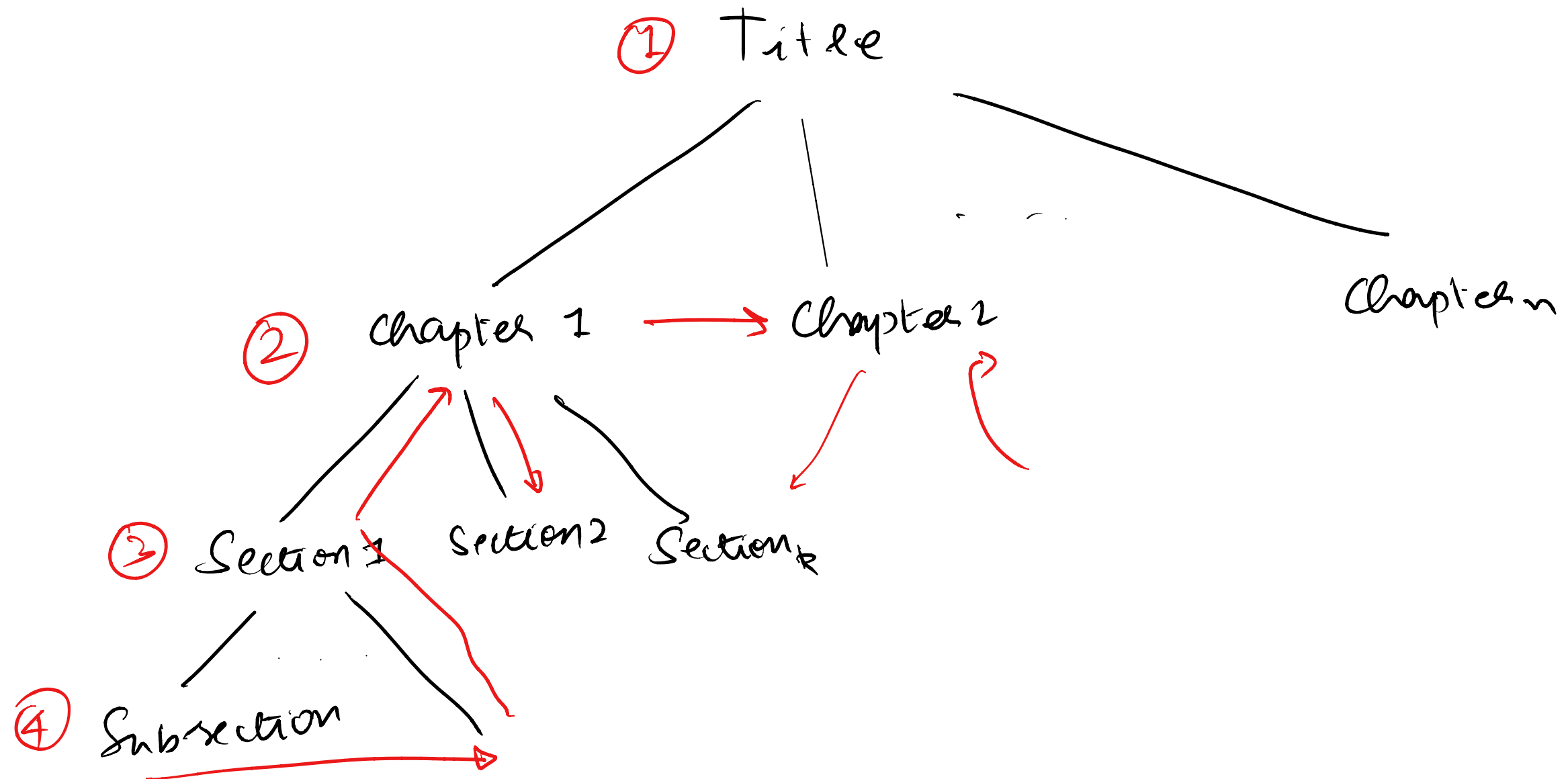


Directory Structure



Post order: calculating memory/space occupied by each file and directory

Book

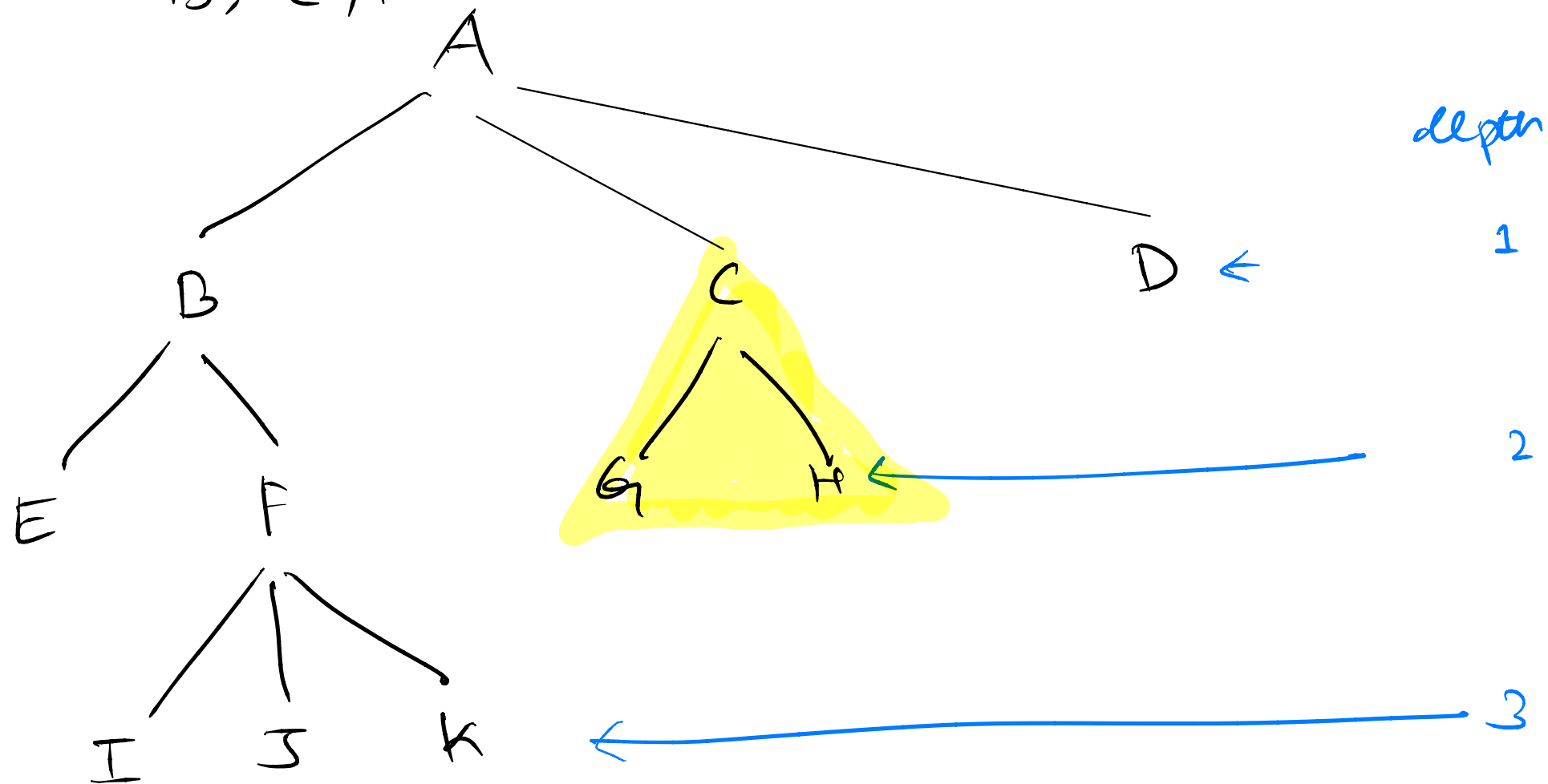


Preorder : Note, followed by children

Terminology

- Root Node: A (has no parent)
- Leaf Nodes: Have no children: E, I, J, K, G, H, D
(external)
- Internal Nodes: Have one or more children
B, C, F

parent
grand parent
child
grand child etc

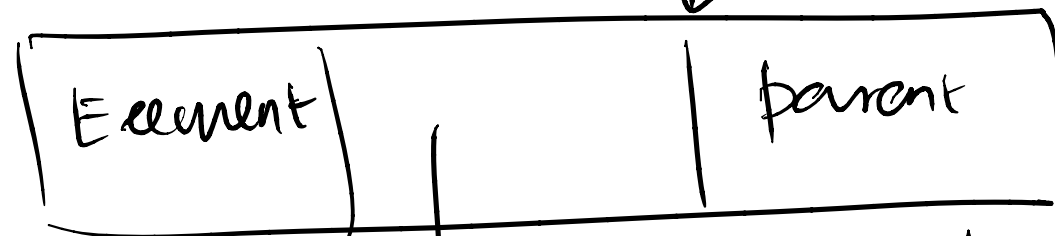


- Depth of a Node: Number of ancestors
- Height of a tree: Maximum Depth of any node
- Subtree: Select a node and pluck out the tree rooted at that node

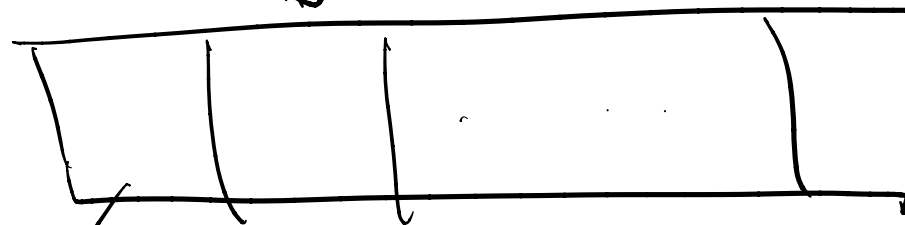
Data Structure

Node in a Tree

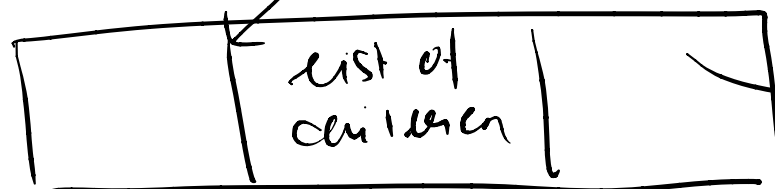
Parent Node



list pointing to children



Child Node



Root Node

Element	list of children	NULL
---------	---------------------	------

Preorder Traversal

visit (node)

for each child in children (node)

visit (child)

Postorder Traversal

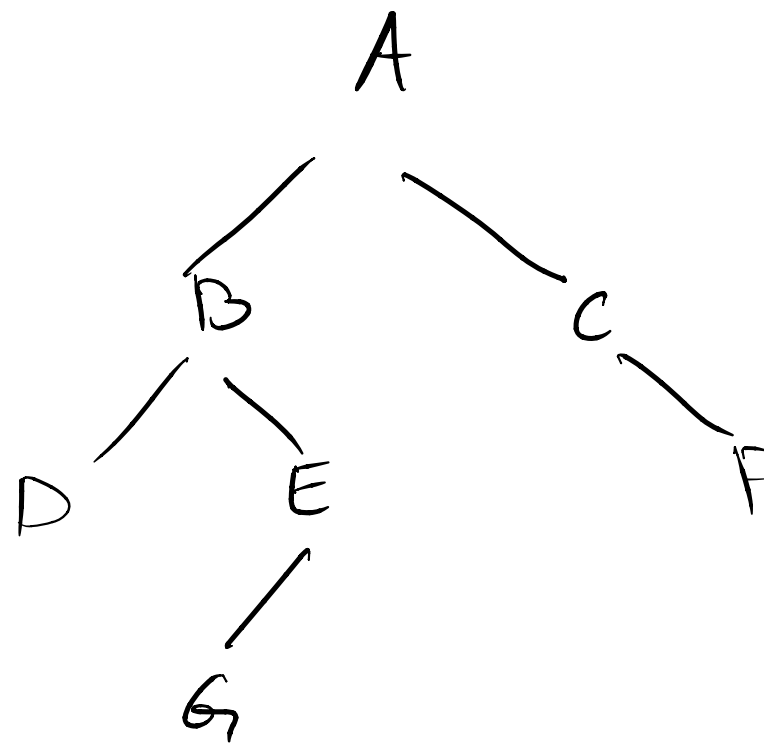
for each child in children (node)

visit (child)

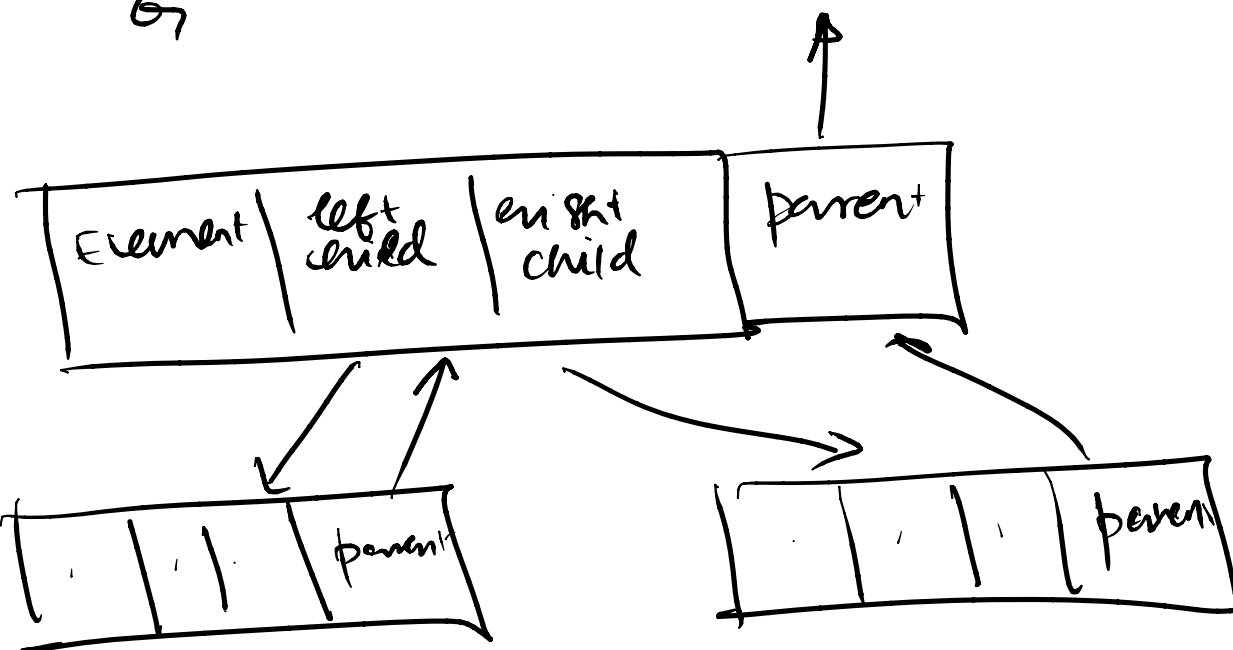
visit (node)

Binary Trees:

- Each Node has at max two children
 - left child
 - right child

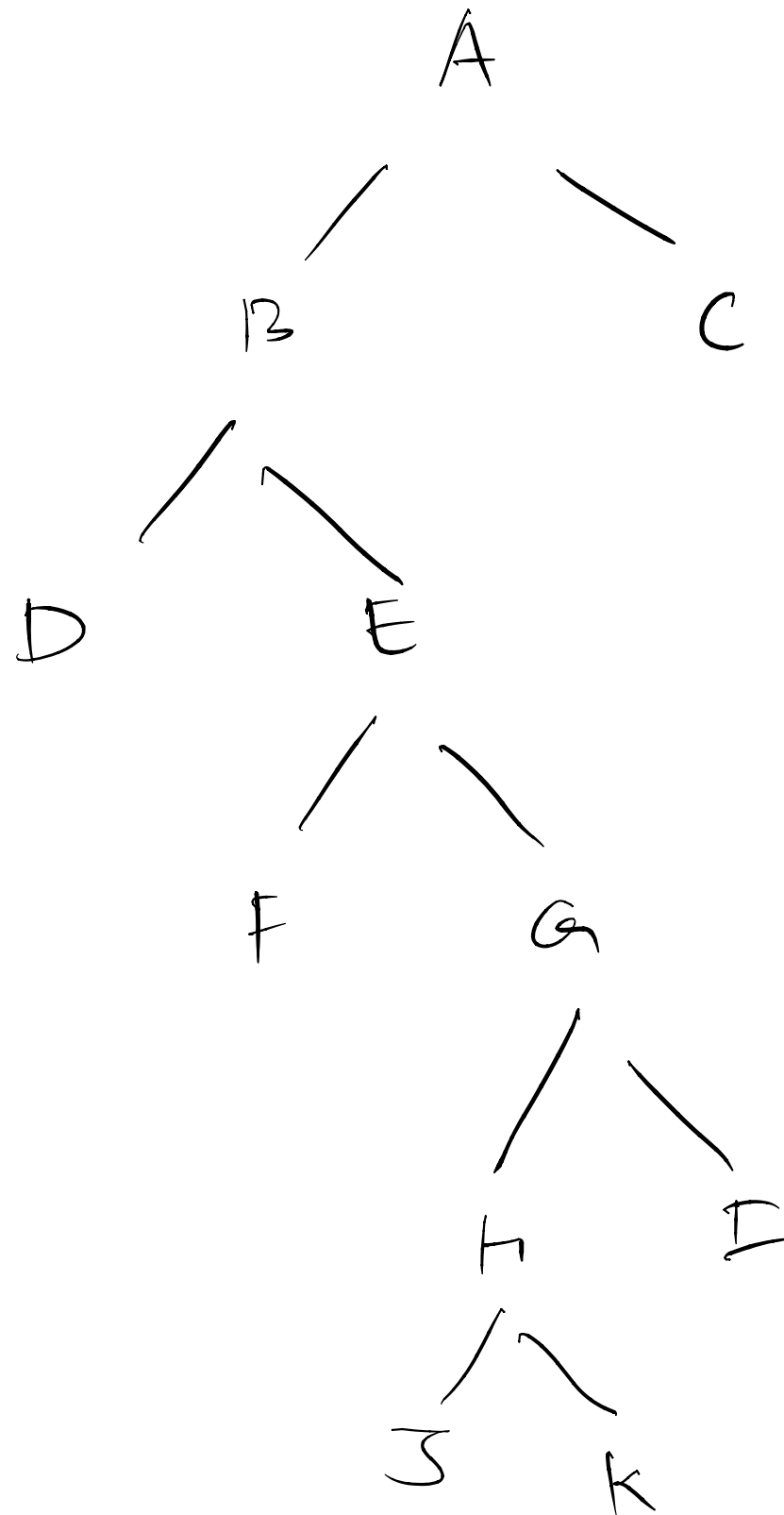


Parent Node



Child Node

- Proper Binary Tree: Either 0 children or 2 children



n : # total nodes, e : # external nodes, i : # internal nodes

h : height

Proper Binary Tree

$$e = i + 1$$



leaf
node



has children

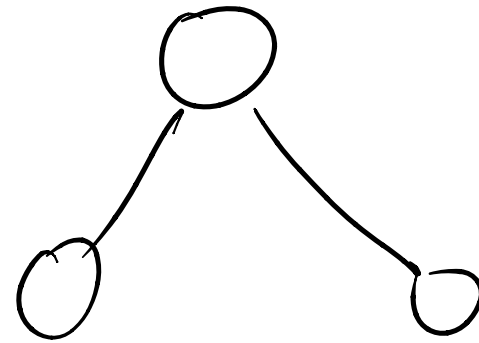
no
children

Tree with root



$$e = 1, i = 0$$

grow this tree



$$e \leftarrow e - 1, i \leftarrow i + 1$$

$$e \leftarrow e + 2$$

- $n = e + i$

$$= e + e - 1$$

$$= 2e - 1$$

- $\frac{n+1}{2} = e$

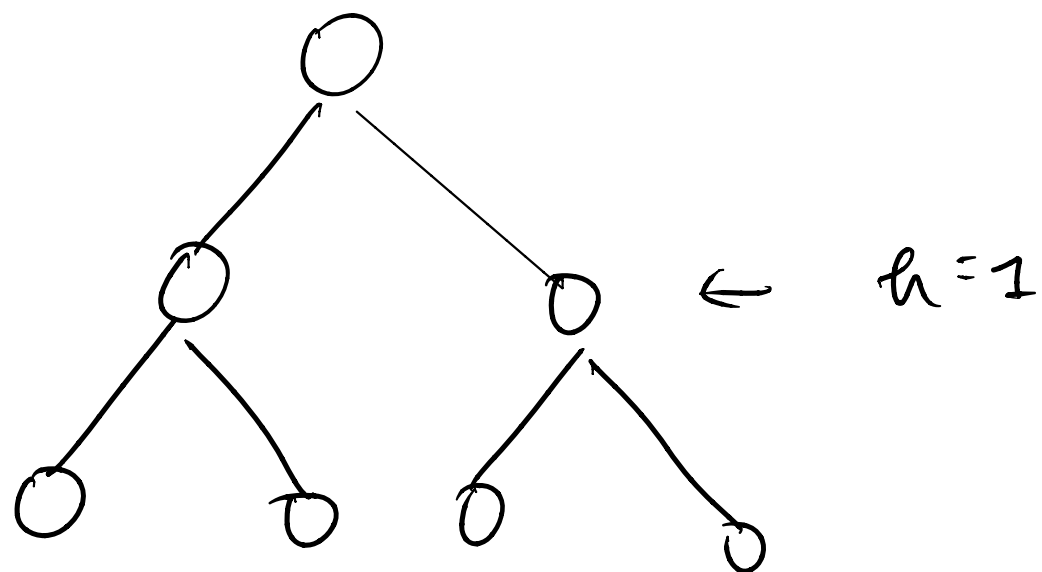
- $n = i + 1 + i$

$$= 2i + 1$$

$$\frac{n-1}{2} = i$$

- $h \leq i$ (loose upper bound)

- $h \leq (n-1)/2$

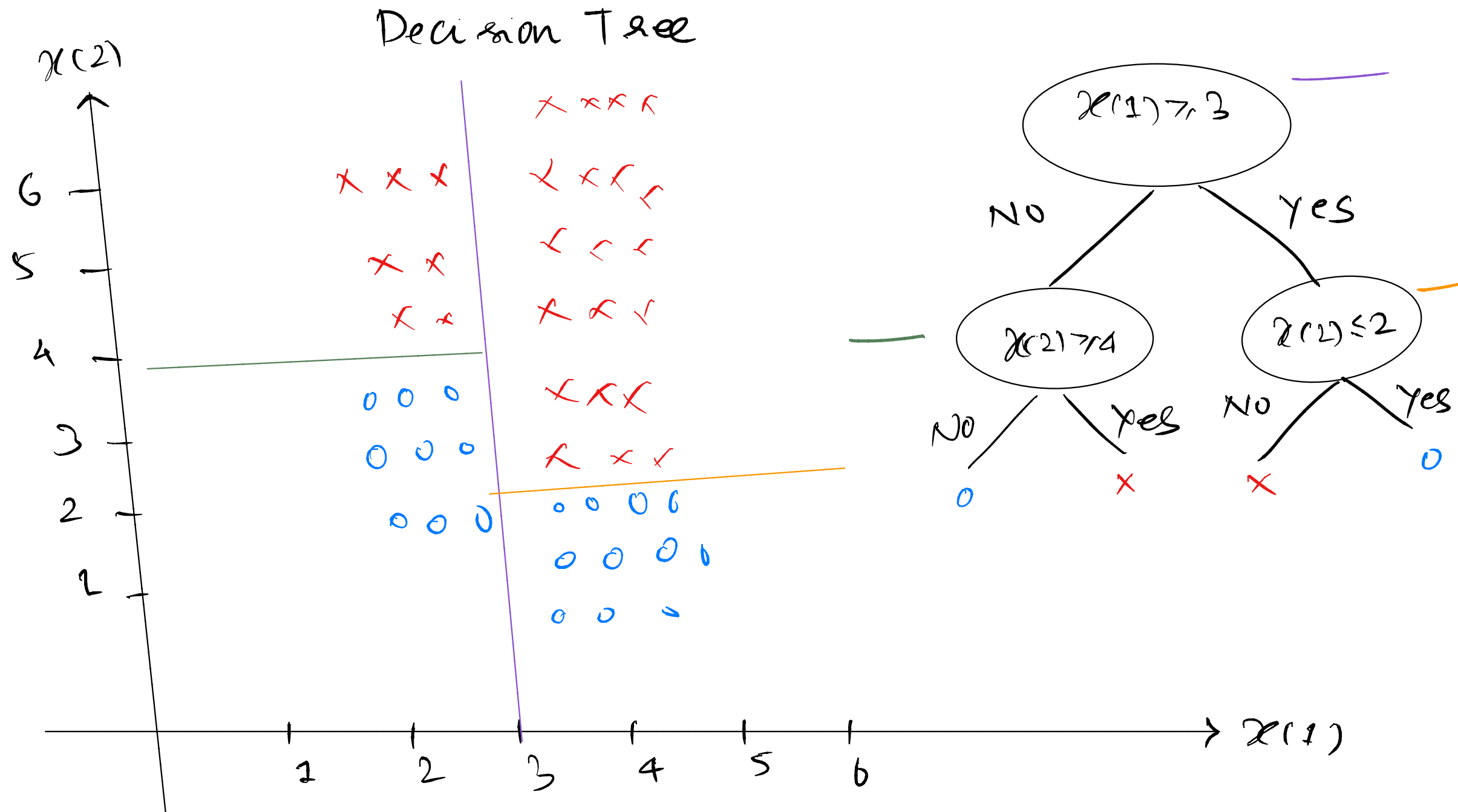


- $e \leq 2^h$
 $\downarrow$ invert

- $h \geq \log_2 e$

- $h \geq \log_2 \frac{n+1}{2}$
 $\stackrel{(a)}{=} \log_2 (n+1) - 1$

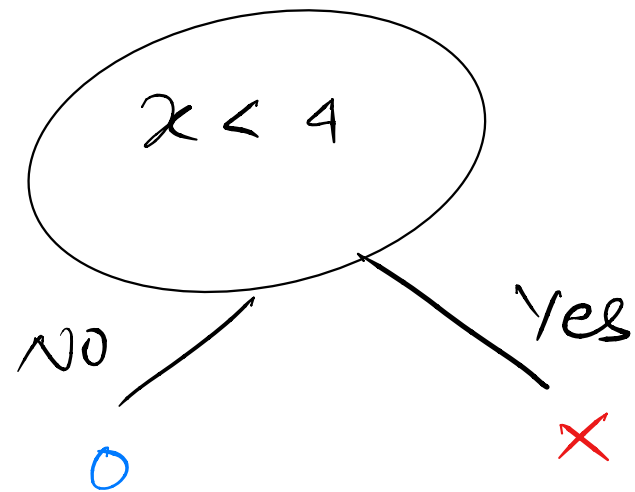
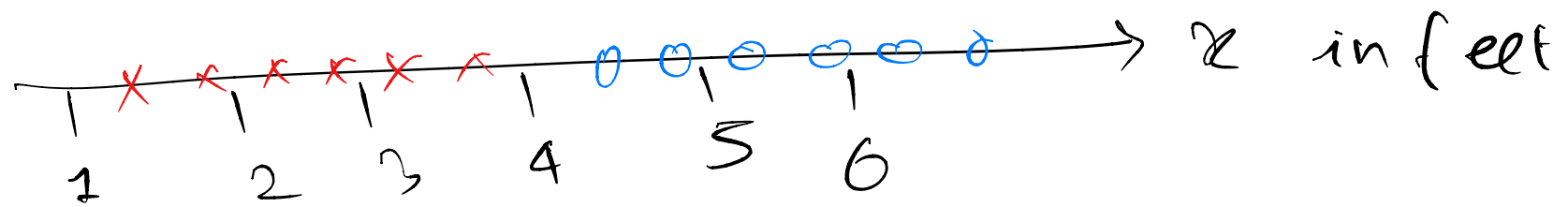
$$h \geq \log_2 (n+1) - 1$$



Exercise: Create a Data Set which is labelled by a binary tree of height 'h'

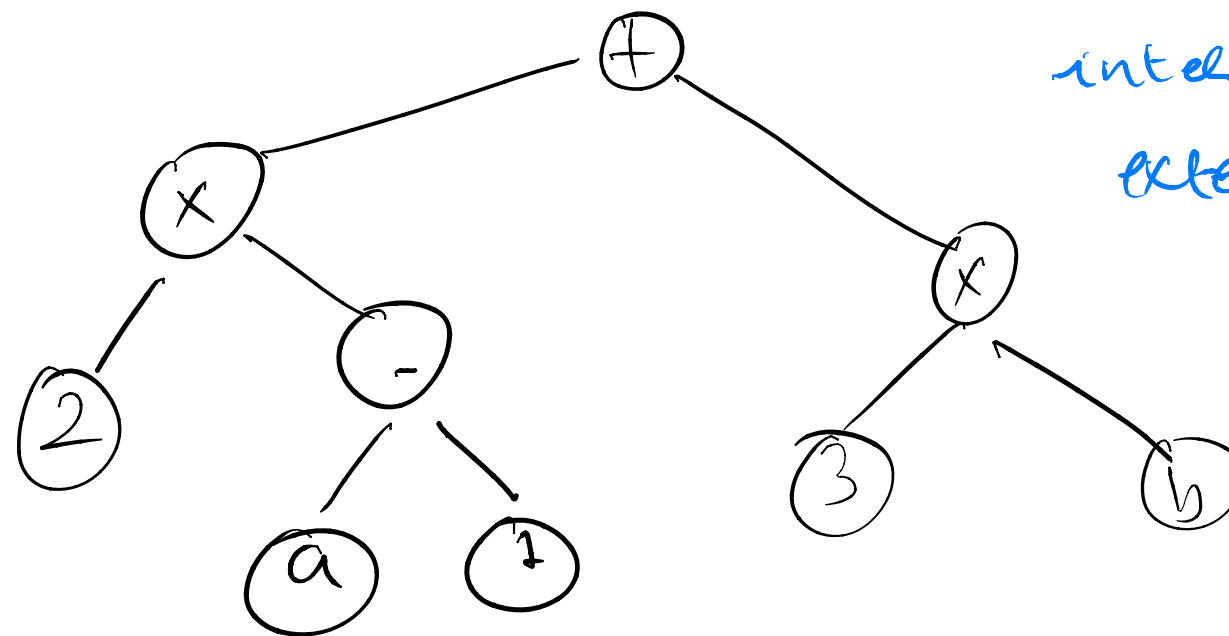
$\times$: kids

0 : adults



Arithmetic Expression Tree

$$(2 \times (a - 1) + (3 \times b))$$



internal node: operators
external: operands

Inorder Traversal

left, node, right

inorder(left(node))
visit(node)
inorder(right(node))

Pre order Traversal

node, left, right

Post order

left, right, node

Printing Arithmetic expression

- print (before left sub-tree
- ") after right sub-tree

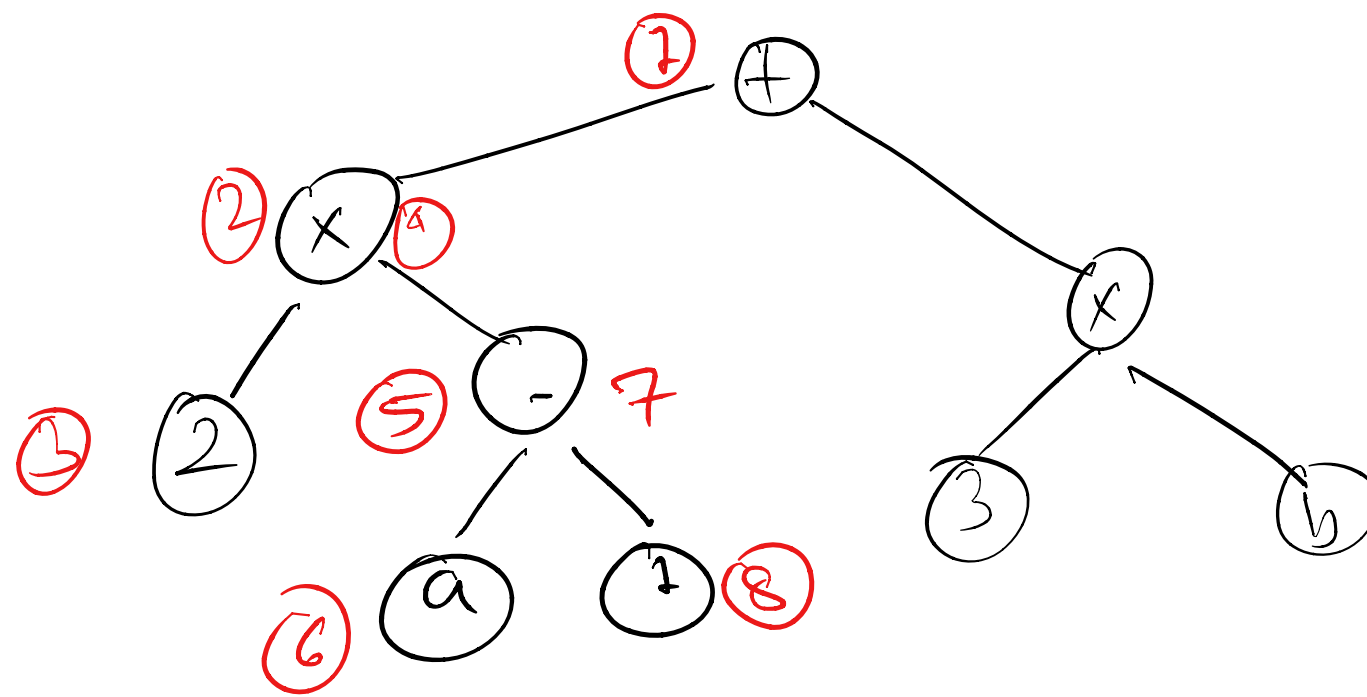
```
if left(node)  $\neq$  NULL  
  print '('
```

```
  inorder(left(node))
```

```
  print (element(node))
```

```
  if right(node)  $\neq$  NULL  
    inorder(right(node))
```

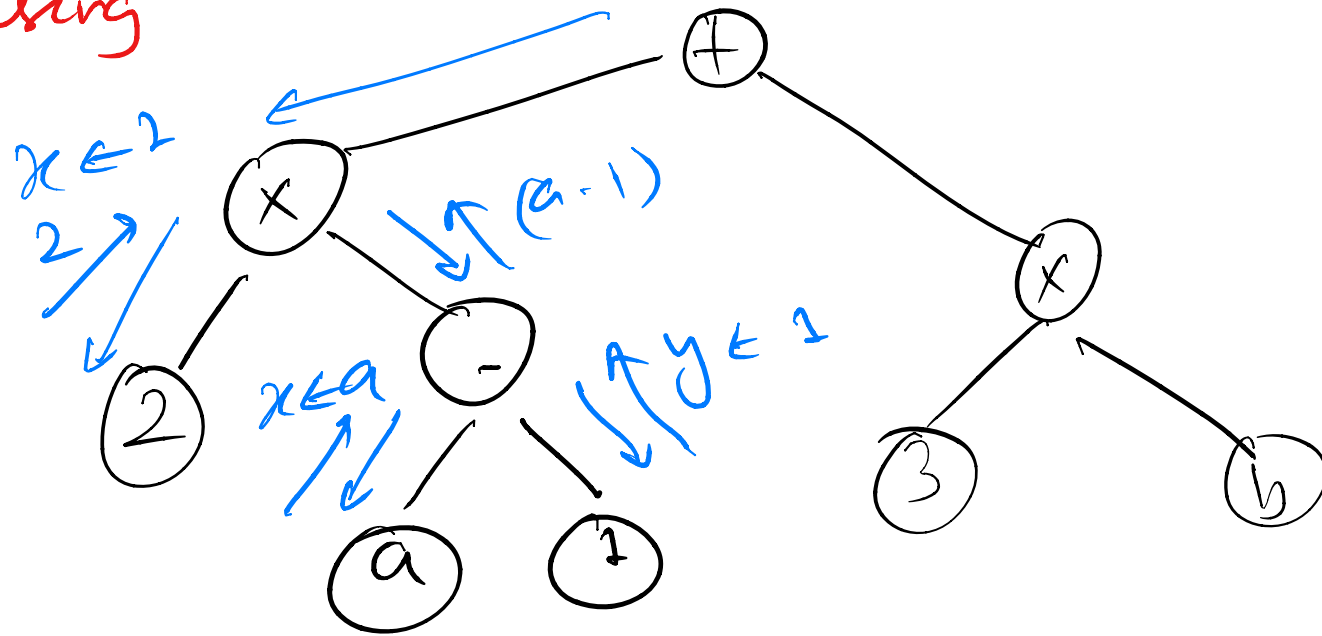
```
  print ')'
```



((2 x (a - 1)
 1 2 3 4 5 6 7 8 9

Evaluate using

Post order



evaluate (node)

if (node is external)

return element @ node

$x \leftarrow \text{evaluate}(\text{left}(\text{node}))$

$y \leftarrow \text{evaluate}(\text{right}(\text{node}))$

return x operator @ node y

Scientists study whales from space

Given Preorder and Inorder traversal, reconstruct a binary tree

Preorder (node, left, right)

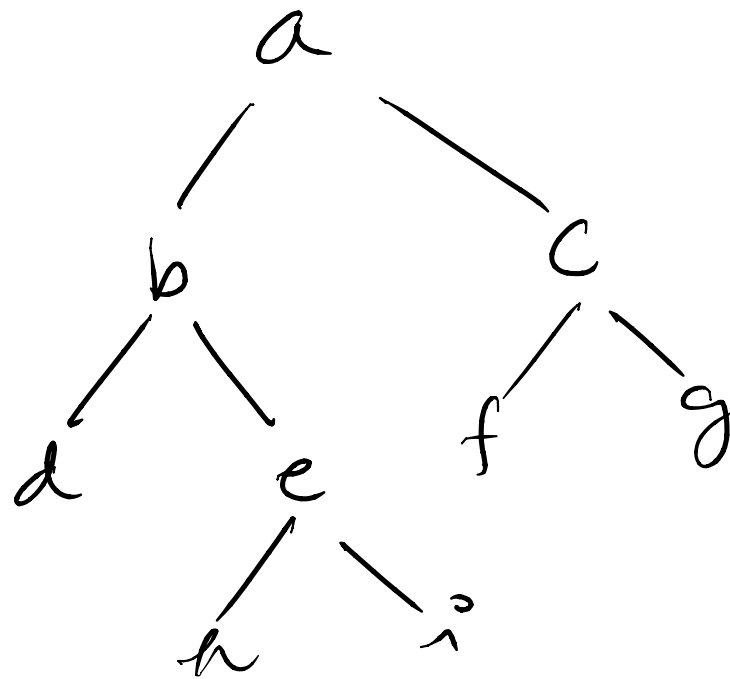
a b d e h i c f g

Inorder (left, node, right)

d b h e i a f c g
Left subtree Right subtree

Fact 1: First element of preorder is a

Fact 2:



Preorder and Post order are given :

Preorder (node, left, right)

a b e i c

Fact 1: a is root

b e i c

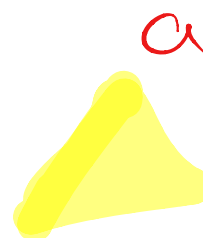
X Case 1: all on left

Case 2: " right

Case 3: either side

Postorder (left, right, node)

i e b c a



Pre order says b is the candidate for root of left subtree

According to post order c is the candidate for root of left subtree